Simulação e Teste de Software

Laboratório 7: Teste Combinatorial e Property-Based

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Laboratório 7

Contexto

Objetivo desta aula

Aplicar as técnicas da aula 7 a problemas novos: conferir conjuntos *pairwise* em código, comparar o *pairwise* com a execução exaustiva para achar defeitos de interação e usar o Hypothesis para achar defeitos com propriedades. Dois exercícios de extensão ficam para casa.

Preparação do Ambiente: Linux

centro universitário FEI

Código

Crie uma pasta `lab07` e, dentro dela, o ambiente virtual e as bibliotecas do laboratório:

Terminal

```
python3 -m venv .venv
source .venv/bin/activate
pip install pytest hypothesis allpairspy
```

Cada exercício tem um arquivo de código e um de testes (`test_*.py`); com o ambiente ativo, rode tudo com `pytest -q`.

Preparação do Ambiente: Windows

</> Código

Os mesmos passos, no PowerShell:

PowerShell

```
python -m venv .venv
.venv\Scripts\Activate.ps1
pip install pytest hypothesis allpairspy
```

- No Prompt de Comando (cmd), a ativação é `.venv\Scripts\activate.bat`
- Se o PowerShell bloquear a ativação, rode antes `Set-ExecutionPolicy -Scope CurrentUser RemoteSigned`
- Com o ambiente ativo, rode os testes com `pytest -q`, como no Linux

Entrega

Contexto

Como entregar

Ao final do laboratório, compacte todos os arquivos `.py` produzidos (código e testes) em um único `.zip` e envie pelo *link* desta atividade no **Moodle**.

Dicas: Recursos do Laboratório

centro universitário FEI

Código

Recursos que talvez sejam novos

```
from itertools import product, combinations
from collections import deque
from allpairspy import AllPairs
import hypothesis.strategies as st

list(product([1, 2], "ab"))       # [(1,'a'), (1,'b'), (2,'a'), (2,'b')]
list(combinations(range(3), 2))   # [(0, 1), (0, 2), (1, 2)]: pares sem ordem
product(*PARAMS)                  # o * passa cada lista como um argumento
list(AllPairs(PARAMS))            # conjunto pairwise; cada caso e uma lista
st.tuples(st.integers(0, 50), st.integers(0, 50))   # gera tuplas (a, b)
deque([1, 2]).popleft()           # 1: fila, sai pelo inicio (extensao)
```

`AllPairs` devolve listas e `product`, tuplas: compare pares de valores, não casos inteiros.

Pairwise e Interações



Envio de Encomendas

Código do Exercício 1

Um sistema de envio de encomendas aceita quatro parâmetros: transportadora, embalagem, seguro e tipo de entrega.

modelo.py

```
PARAMS = [
    ["Correios", "Jadlog", "Loggi"],
    ["caixa", "envelope", "tubo"],
    ["com seguro", "sem seguro"],
    ["normal", "expressa", "agendada"],
]
```

Na aula 7, o *pairwise* foi conferido pela contagem de pares à mão; aqui a conferência vira código.

Envio de Encomendas

Exercício 1

Com os parâmetros de `modelo.py`:

a) Quantas combinações completas existem? Qual é o número mínimo de casos de um conjunto *pairwise*, e por quê?

b) Gere o conjunto *pairwise* com o allpairspy e confira a quantidade de casos

c) Escreva `pares_de(casos)`, que devolve o conjunto de pares de valores cobertos por uma lista de casos, e um teste que confirma que o conjunto gerado cobre todos os pares do produto completo

d) Escreva um teste que mostre que o verificador funciona: com apenas 3 casos do conjunto, ele precisa apontar pares descobertos

Frete com Defeitos

Exercício 2

Especificação do cálculo de frete:

- Valor base pelo peso: leve R$10, médio R$20, pesado R$40
- Região norte: mais R$8
- Entrega expressa: multiplica o valor por 1,5
- Seguro: mais R$5, somado ao final

Os parâmetros, nesta ordem: peso (`"leve"`, `"medio"`, `"pesado"`), urgência (`"normal"`, `"expressa"`), seguro (`False`, `True`) e região (`"sul"`, `"sudeste"`, `"norte"`).

Frete com Defeitos

Código do Exercício 2

frete.py

```python
def frete(peso, urgencia, seguro, regiao):
    base = {"leve": 10, "medio": 20, "pesado": 40}[peso]
    if regiao == "norte" and peso != "leve":
        base += 8
    if urgencia == "expressa":
        base = base * 1.5
        if base > 60:
            base = 60
    if seguro:
        base += 5
    return round(base, 2)
```

Esta implementação foi escrita por outra pessoa e tem defeitos.

Frete: Tarefas

Exercício 2

a) Escreva `esperado(peso, urgencia, seguro, regiao)` a partir da especificação
b) Rode um teste parametrizado com o conjunto *pairwise*, gerado com os parâmetros na ordem dada, e anote as falhas
c) Rode o mesmo teste com as 36 combinações (`itertools.product`) e anote as falhas
d) Para cada defeito, diga o grau da interação[1] e se o *pairwise* o encontrou. Por que encontrou, ou por que não?
e) Corrija a função e confirme que todos os casos passam

[1] *Grau da interação: número de parâmetros que precisam ter valores específicos ao mesmo tempo para o defeito causar a falha.*

Propriedades



Mesclar Intervalos

</> Código do Exercício 3

A função `mesclar` une intervalos que se sobrepõem: `[(1, 3), (2, 6), (8, 10)]` vira `[(1, 6), (8, 10)]`.

intervalos.py

```
def mesclar(intervalos):
    resultado = []
    for ini, fim in sorted(intervalos):
        if resultado and ini <= resultado[-1][1]:
            resultado[-1] = (resultado[-1][0], fim)
        else:
            resultado.append((ini, fim))
    return resultado
```

A função tem um defeito, e não há exemplos prontos: use propriedades para achá-lo.

Mesclar Intervalos

Exercício 3

a) Gere as entradas com `st.lists(st.tuples(...))` de inteiros de 0 a 50 e escreva `normalizar`, que põe cada par na forma `(ini, fim)` com `ini <= fim`

b) Escreva três propriedades de `mesclar`: idempotência; resultado em ordem e sem sobreposição; todo intervalo da entrada está contido em algum intervalo do resultado

c) Rode: quais propriedades falham? Qual é o contraexemplo minimizado? Explique o defeito

d) Corrija a função e rode de novo. Qual entrada o Hypothesis testa primeiro, e por quê?

Síntese

- Conferir a cobertura vira código: com `pares_de`, o conjunto gerado é medido contra o universo de pares
- O *pairwise* garante os defeitos de grau 2; defeitos de grau 3 ou mais podem escapar, e a execução exaustiva, quando cabe, mostra o que ficou de fora
- Propriedades sobre entradas geradas acham defeitos que nenhum exemplo previu, e várias propriedades juntas especificam melhor que uma só
- Para casa: o gerador guloso (exercício 4) e um oráculo para sequências de operações (exercício 5)

Extensões: para Casa



Um Gerador *Pairwise* Próprio (Extensão)

Exercício 4

Na aula 7, o conjunto das 12 combinações foi montado à mão por um algoritmo guloso. Aqui ele vira código.

a) Escreva `gerar_pairwise(params)`: a cada passo, escolha entre as combinações completas a que cobre mais pares ainda não cobertos, até não faltar nenhum; use `pares_de` do exercício 1

b) Teste que o conjunto gerado cobre todos os pares no modelo do exercício 1 e nas dez chaves liga/desliga da aula 7

c) Compare, nos dois modelos, o número de casos do seu gerador, do allpairspy e o mínimo possível

d) Por que nenhum dos dois geradores garante o mínimo? O que acontece com o seu gerador com 20 chaves?

Fila Circular: Código (1/2)

Código do Exercício 5

fila.py

```
class FilaCircular:
    def __init__(self, capacidade):
        self.itens = [None] * capacidade
        self.inicio = 0
        self.tamanho = 0

    def enfileirar(self, x):
        if self.tamanho == len(self.itens):
            raise OverflowError("fila cheia")
        fim = self.inicio + self.tamanho
        self.itens[fim] = x
        self.tamanho += 1
```

Fila Circular: Código (2/2)

Código do Exercício 5

fila.py (continuação)

```
def desenfileirar(self):
    if self.tamanho == 0:
        raise IndexError("fila vazia")
    x = self.itens[self.inicio]
    self.inicio = (self.inicio + 1) % len(self.itens)
    self.tamanho -= 1
    return x
```

Uma fila circular[1] guarda até capacidade itens. Ela tem um defeito que exemplos escritos à mão dificilmente pegam.

[1] *Fila circular: fila de capacidade fixa guardada em um vetor; o início e o fim voltam à primeira posição do vetor ao chegar à última.*

Fila Circular (Extensão)

Exercício 5

Na aula 7, o padrão oráculo comparou uma função com uma implementação confiável. Aqui o oráculo é uma `deque`[1], e a entrada gerada é uma sequência de operações: em uma lista de inteiros de -1 a 9, um valor não negativo enfileira esse valor e -1 desenfileira.

a) Escreva uma propriedade que aplique a sequência à fila, de capacidade 3, e à `deque`, e confira a cada operação o valor desenfileirado, o tamanho e as exceções de fila cheia e de fila vazia

b) Rode, leia o contraexemplo minimizado e explique o defeito

c) Corrija a fila e rode de novo

[1] *deque: fila de duas pontas da biblioteca padrão do Python, com `append` e `popleft`.*

Simulação e Teste de Software

Laboratório 7: Teste Combinatorial e Property-Based

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026